



LINEミニアプリでサービスメッセージを送信してみよう

2026/03/13 LINEミニアプリ Tech Meetup #4

クラスメソッド株式会社 リテールアプリ共創部

高垣 龍平

自己紹介



- 所属
 - クラスメソッド株式会社
 - リテールアプリ共創部 マッハチーム
 - 2024年度 新卒入社
- 名前
 - 高垣龍平
 - X:るおん (@ruonp24)

経歴

classmethod

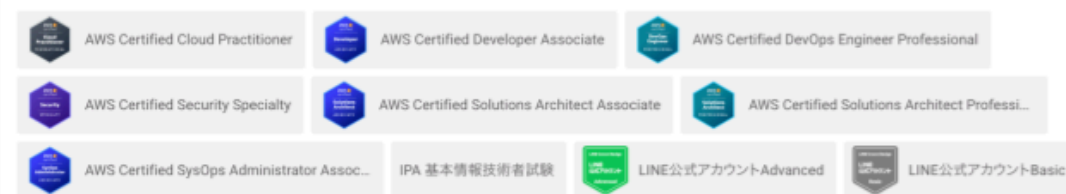
- 大阪大学
 - 文系（中国語専攻）
- 学生時代：アパレル系自社開発企業（インターン）
 - toC向けWebアプリ
- 2024年4月～：クラスメソッド株式会社
 - Webアプリ新規開発・運用保守
 - LINEミニアプリ開発

技術スタック

- フロントエンド（React/Next.js）
- バックエンド（RoR/Lambda/Express/Next.js）
- インフラ（AWS）
- LINE

実績

- LINE API Expert
- 2025 Japan AWS Jr. Champions



1. サービスメッセージとは
2. LINE公式アカウントからの配信との違い
3. 全体像
4. 実装してみた
5. 注意点・まとめ

サービスメッセージとは

ユーザーの操作に対する確認・応答として 送信される通知機能

例えば...

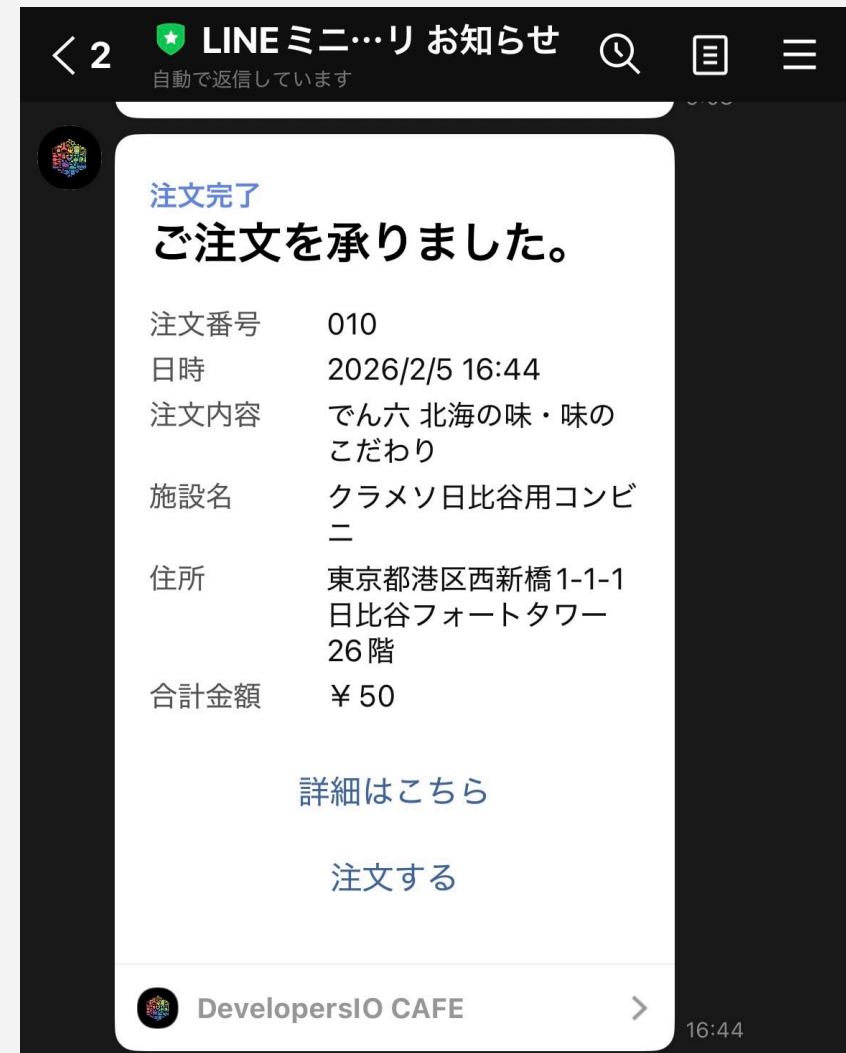
- 予約完了の通知
- リマインド
- 会員登録完了のお知らせ

表示場所

LINEアプリ内の専用トークルーム

「LINEミニアプリ お知らせ」に表示される

認証済みのLINEミニアプリ専用の機能

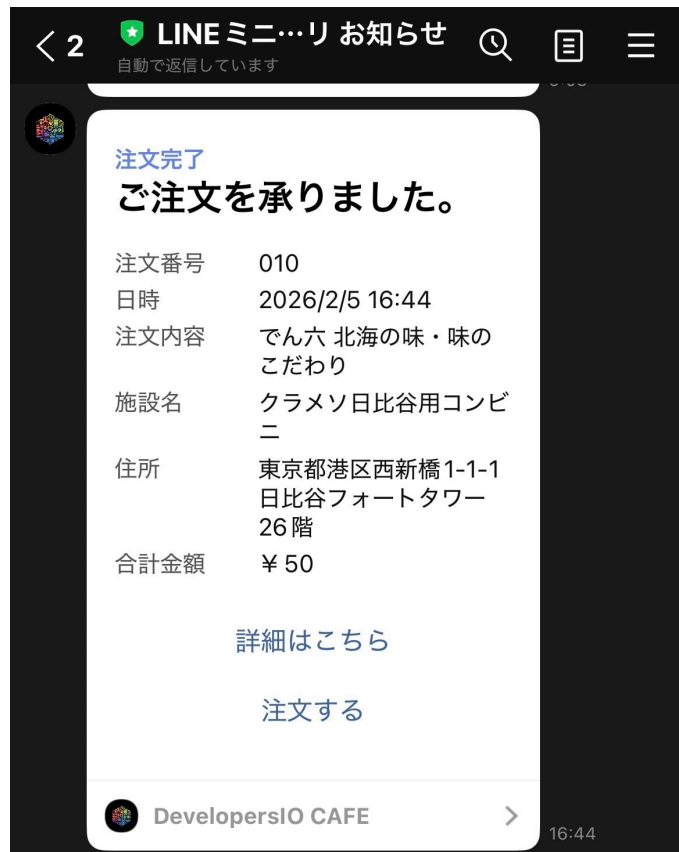


LINE公式アカウントからの配信との違い

LINE公式アカウントからの配信との比較

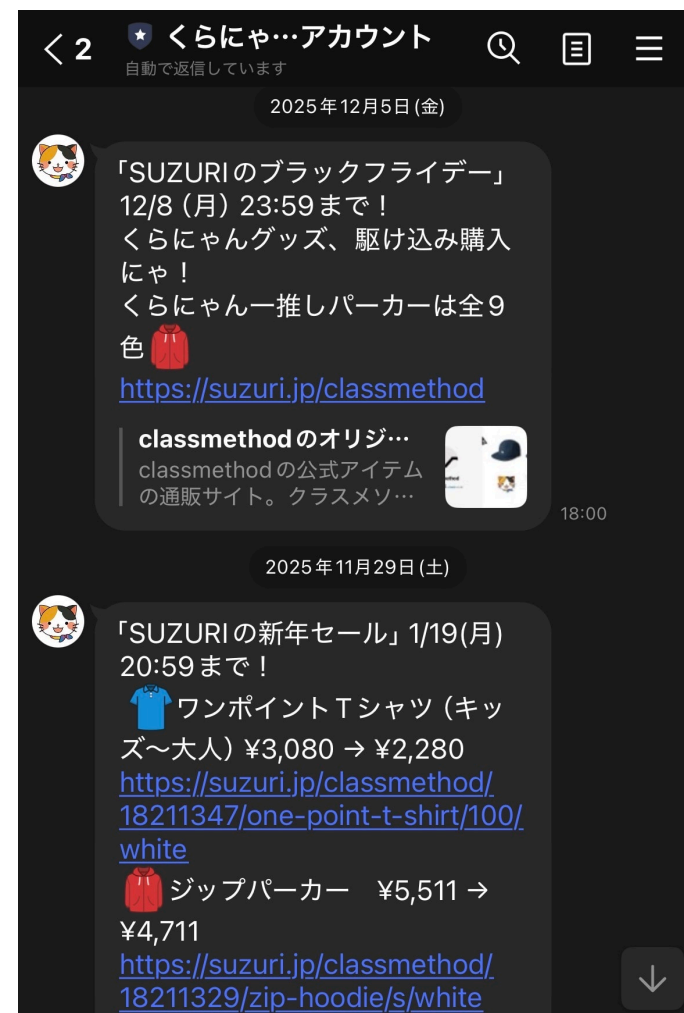
項目	サービスメッセージ	LINE公式アカウントからの配信
料金	無料	従量課金制
送信回数制限	同一操作に対して最大5回まで	プランに応じた制限
メッセージ形式	テンプレート形式のみ	自由形式
表示場所	専用トークルーム（LINEミニアプリ お知らせ）	公式アカウントのトーク
対象	認証済みLINEミニアプリのみ	LINE公式アカウント

サービスメッセージ



専用トークルーム「LINEミニアプリ お知らせ」

LINE公式アカウントからの配信



公式アカウントのトーク

サービスメッセージの4つの特徴

1. 無料で利用可能

LINE公式アカウントからの配信は送信数に応じて課金されるが、サービスメッセージは**無料**

2. 送信回数の制限

同じ操作に対して**最大5回まで**。スパム的な利用を防止

3. テンプレート形式

LINEヤフー社が提供する**審査済みテンプレート**を使用。ユーザー体験の一貫性を担保

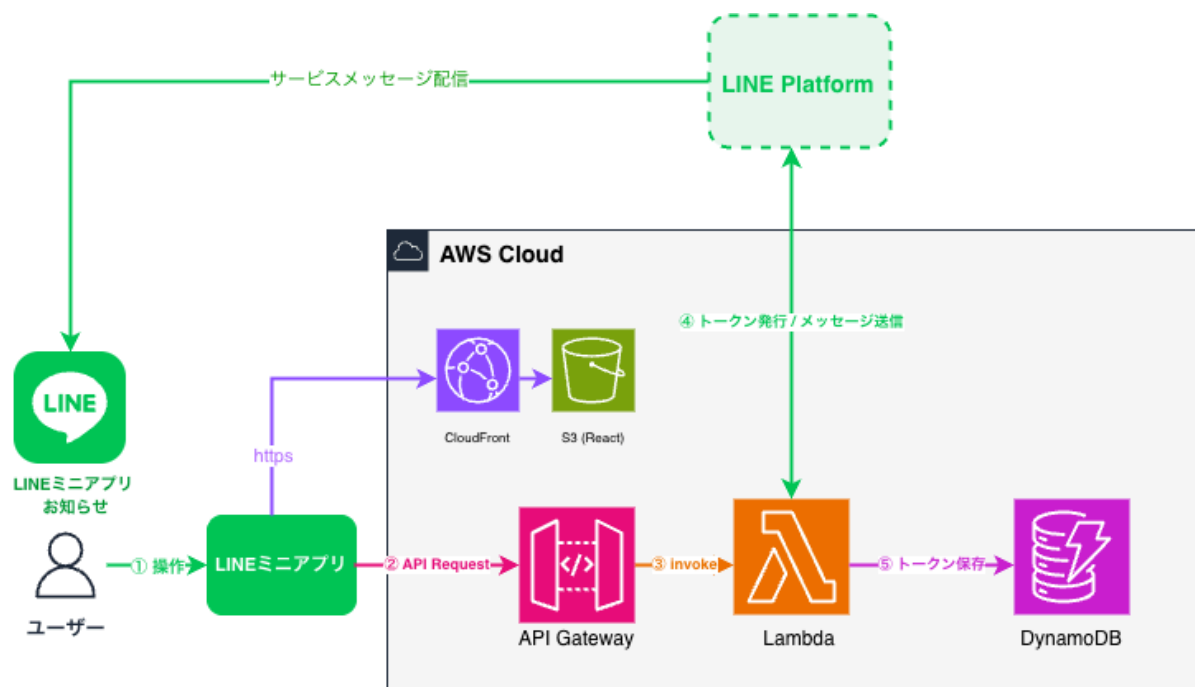
4. ブロック中でも送信可能

LINE公式アカウントがブロックされていても、専用トークルーム「LINEミニアプリ お知らせ」に**配信可能**

全体像

- フロントエンド : React + TypeScript (S3 + CloudFront)
- バックエンド : Express.js + TypeScript (Lambda + API Gateway)
- データベース : DynamoDB

フロントエンドのLINEミニアプリから**LIFFアクセストークン**を含めてAPIリクエストを送信し、バックエンドで**サービス通知トークンの発行とサービスメッセージの送信**を行う



実装してみた



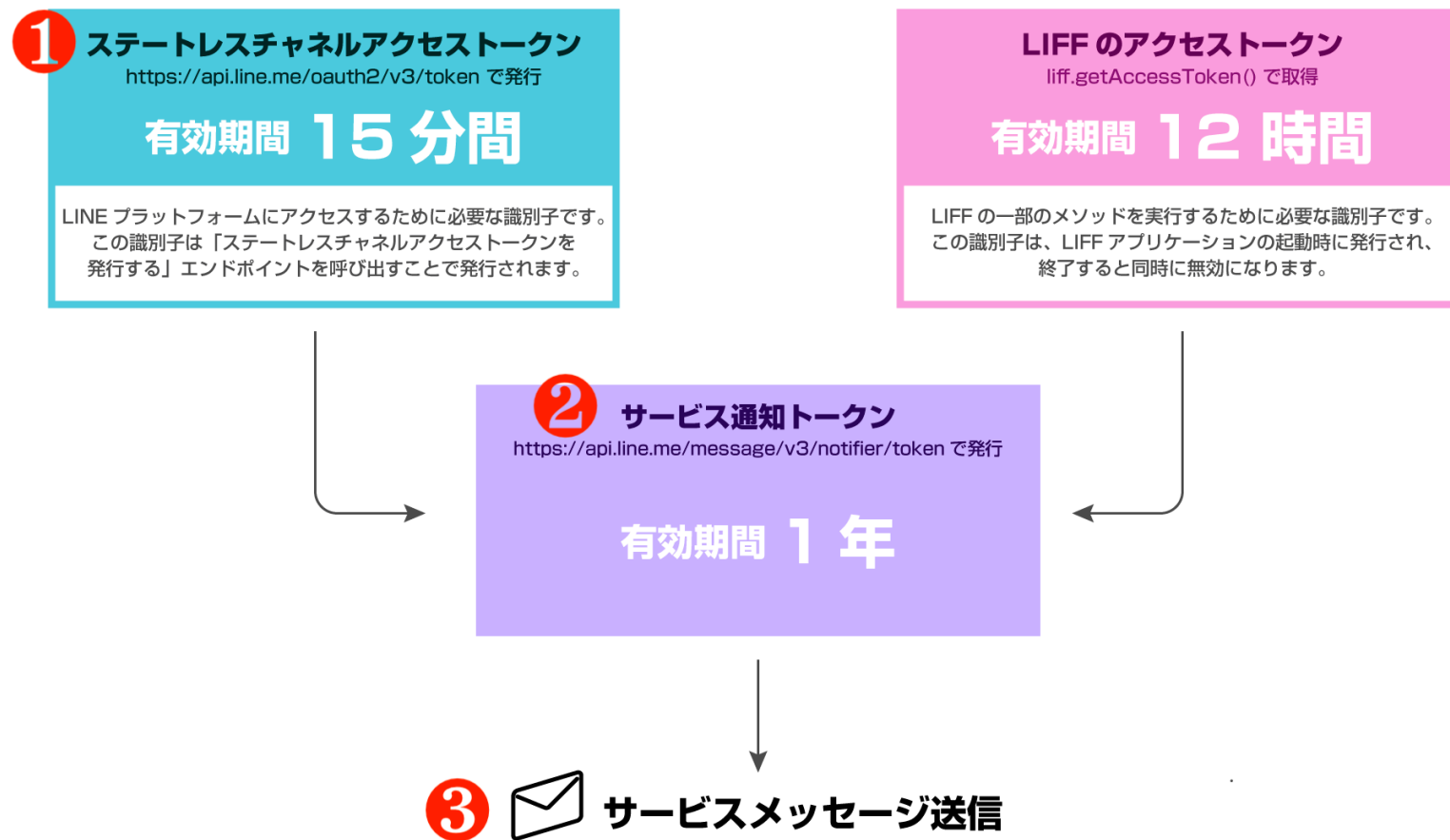
事前準備

1. LINE DevelopersコンソールでチャンネルID・チャンネルシークレットを取得
2. サービスメッセージテンプレートを登録
3. LIFFアプリの設定確認

処理の流れ（公式ドキュメントより）

14

サービスメッセージの送信には、ステートレスチャネルアクセストークンとLIFFアクセストークンの2つが必要



LINE Developersコンソールでの設定手順

1. 対象チャネルの「サービスメッセージ」タブを開く
2. 「テンプレートを追加」をクリック
3. テンプレートカテゴリを選択
4. テンプレートの言語を選択（6言語対応）
5. テンプレート名を設定

ポイント

- テンプレートは**LINEヤフー社**が用意したものを使用
- テンプレート内の変数（`${number}`，`${reg_date}` 等）はAPI呼び出し時に**任意の値**を設定可能

サービスメッセージテンプレートを追加する

未認証ミニアプリでは、開発用LINEミニアプリでのみ利用可能です。

テンプレートを選擇する

カテゴリ Order > 言語 日本語 > テンプレート名 Back-in-stock notification (detailed)

テンプレート詳細

API用テンプレート名 restock_d_ja

Title

項目	値
Title	ご登録の商品が入荷しました。
Subtitle	入荷通知

Detail

ラベル	値
登録番号	<code>\${number}</code>
登録日時	<code>\${reg_date}</code>
商品名	<code>\${product_name}</code>
施設名	<code>\${shop_name}</code>
住所	<code>\${address}</code>
入荷日時	<code>\${instock_date}</code>
商品価格	<code>\${price}</code>

Button

ラベル	URL
詳細はこちら	<code>\${btn1_url}</code>

プレビュー

入荷通知
ご登録の商品が入荷しました。

登録番号 1357
登録日時 2033/8/22 0:00
商品名 Tシャツ
施設名 ブラウン新宿店
住所 東京都新宿区新宿4-1-6
入荷日時 2033/8/22 0:00
商品価格 400円

[詳細はこちら](#)
必ずお読みください
[変更する](#)
[地図はこちら](#)

Service Messages >

LIFFアクセストークンをバックエンドに送信

```
const sendServiceMessage = async (params: Record<string, string>) => {
  const liffAccessToken = liff.getAccessToken();

  if (!liffAccessToken) {
    throw new Error("LIFFアクセストークンが取得できませんでした");
  }

  const response = await fetch("https://your-api-endpoint.com/send-message", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-LIFF-Access-Token": liffAccessToken,
    },
    body: JSON.stringify({ params }),
  });

  return response.json();
};
```

ポイント

`liff.getAccessToken()` で取得したLIFFアクセストークンを `X-LIFF-Access-Token` ヘッダーに設定

Lambda関数のハンドラー概要

```
export const handler = async (event: APIGatewayProxyEvent) => {
  const liffAccessToken = event.headers["x-liff-access-token"];
  const { userId, params } = JSON.parse(event.body || "{}");

  // 1. チャンネルアクセストークンを発行
  const channelAccessToken = await getChannelAccessToken();

  // 2. サービス通知トークンを発行
  const tokenResponse = await issueServiceNotificationToken(
    channelAccessToken, liffAccessToken
  );

  // 3. サービスメッセージを送信
  const sendResponse = await sendServiceMessage(
    channelAccessToken, tokenResponse.notificationToken,
    TEMPLATE_NAME, params
  );

  // 4. サービス通知トークンをDBに保存（再利用のため）
  await tokenRepository.save({
    userId, notificationToken: sendResponse.notificationToken,
    remainingCount: sendResponse.remainingCount,
  });
};
```

LINE APIを呼び出すために必要なトークンを発行

POST https://api.line.me/oauth2/v3/token

```
const getChannelAccessToken = async (): Promise<string> => {  
  const response = await fetch("https://api.line.me/oauth2/v3/token", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/x-www-form-urlencoded",  
    },  
    body: `grant_type=client_credentials&client_id=${CHANNEL_ID}&client_secret=${CHANNEL_SECRET}`,  
  });  
  
  const data = await response.json();  
  return data.access_token;  
};
```

ポイント

- チャネルIDとチャネルシークレットは環境変数から読み込み

Step 2 : サービス通知トークンの発行

LIFFアクセストークンとチャネルアクセストークンで発行

POST https://api.line.me/message/v3/notifier/token

```
const issueServiceNotificationToken = async (
  channelAccessToken: string, liffAccessToken: string
) => {
  const response = await fetch("https://api.line.me/message/v3/notifier/token", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${channelAccessToken}`,
    },
    body: JSON.stringify({ liffAccessToken }),
  });
  return await response.json();
};
```

レスポンスの内容

- `notificationToken` : メッセージ送信に使用するトークン
- `expiresIn` : トークンの有効期限 (最大**1年間**有効)
- `remainingCount` : 残り送信可能回数 (最大**5回**)

Step 3 : サービスメッセージの送信

サービス通知トークンを使用してメッセージを送信

POST https://api.line.me/message/v3/notifier/send?target=service

```
const sendServiceMessage = async (
  channelAccessToken: string, notificationToken: string,
  templateName: string, params: Record<string, string>
) => {
  const response = await fetch(
    "https://api.line.me/message/v3/notifier/send?target=service",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${channelAccessToken}`,
      },
      body: JSON.stringify({ notificationToken, templateName, params }),
    }
  );
  return await response.json();
};
```

ポイント

- `templateName` : LINE Developersコンソールで設定した**テンプレート名**を指定
- `params` : テンプレート内の変数に渡す値を設定

送信後のレスポンスに含まれるトークンをDynamoDBに保存

なぜ保存するのか？

同じLIFFアクセストークンで `issueServiceNotificationToken` を呼べるのは**1回だけ**。
次回以降のメッセージ送信では、保存した `notificationToken` を**再利用**する必要がある。

保存する情報

- `userId`：ユーザーの識別子
- `notificationToken`：次回送信に使うトークン
- `remainingCount`：残り送信可能回数

認証済みミニアプリと未認証ミニアプリ

本番環境（認証済み）

- LINEヤフー社による**審査が必要**
- 審査には数日～数週間
- 実際のユーザーに送信可能
- テンプレートも審査通過が必須

開発環境（未認証）

- **審査なし**で実装・テスト可能
- 開発者・テスターにのみ送信
- 本番リリース前に動作確認
- テンプレートのテストも可能

審査を待たずに、先に実装を完了させることが可能

1. 開発環境で実装・テスト（未認証ミニアプリ）
 - └ 審査なしで動作確認
2. 実装完了後に審査を申請
3. 審査通過後に本番環境へデプロイ

「審査に通るまで実装できない」という心配は不要

注意点

1. 本番環境では認証済みが必須

実際のユーザーへの送信には審査通過が必須。開発環境では未認証でも動作確認は可能

2. テンプレートの事前登録

使用するテンプレートはLINE Developersコンソールで事前登録・審査通過が必要。チャンネルごとに**最大20個**まで

3. 送信回数の制限

同一操作に対して**最大5回**まで。 `remainingCount`を確認して残り送信可能回数を把握

4. サービス通知トークンの再利用

同じLIFFアクセストークンでトークン発行APIを呼べるのは**1回だけ**。レスポンスのトークンを再利用

5. ユーザーアクションが起点

ユーザーがミニアプリ上で行った操作に対してのみ送信可能。バッチ処理や外部からの一方的な通知には**使用不可**

6. プロモーション送信は禁止

値下げやプロモーション情報の送信は**禁止**。ユーザー操作に対する確認・応答のみ

サービスメッセージは無料で送信できる通知機能

LINE公式アカウントの従量課金メッセージと比べ、**無料**で利用可能。ブロック中でも配信できる

3ステップで実装可能

チャネルアクセストークン → サービス通知トークン → メッセージ送信 の流れで実装

用途に応じた使い分けが重要

テンプレート形式のみ・送信回数制限あり。予約通知やリマインドなど**ユーザーにとって重要な情報**を伝える場面で活用

参考リンク

- [サービスメッセージ - LINE Developers](#)
- [サービス通知トークン発行 API](#)
- [サービスメッセージ送信 API](#)

The logo for 'classmethod' is presented within a black-outlined rectangular frame. The frame has a small notch at the top right corner. The word 'classmethod' is written in a bold, black, sans-serif font, centered within the frame.

classmethod

ご清聴ありがとうございました